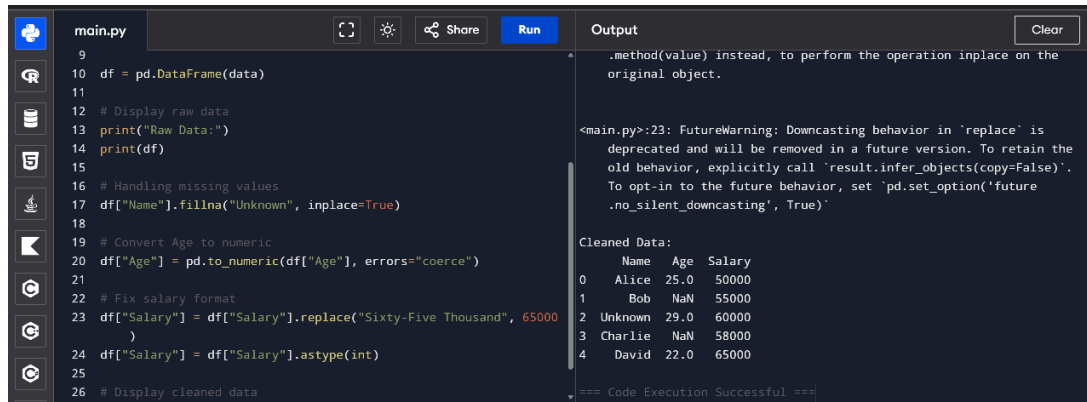


SCENARIO 1



The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code in the notebook performs the following steps:

- Imports pandas as pd.
- Creates a DataFrame from a dictionary 'data'.
- Prints the raw data.
- Handles missing values by filling 'Name' with 'Unknown'.
- Converts 'Age' to numeric, coercing errors.
- Replaces 'Salary' values, specifically 'Sixty-Five Thousand' with 65000.
- Converts 'Salary' to integer type.
- Prints the cleaned data.

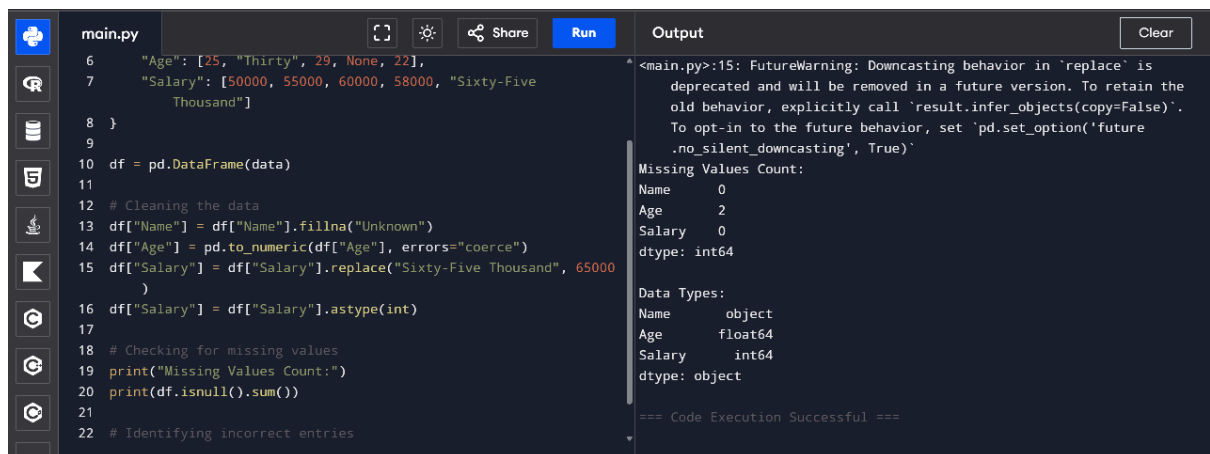
The output of the notebook shows a FutureWarning about the 'replace' method and a table of the cleaned data:

```
Cleaned Data:
```

	Name	Age	Salary
0	Alice	25.0	50000
1	Bob	NaN	55000
2	Unknown	29.0	60000
3	Charlie	NaN	58000
4	David	22.0	65000

The output also indicates that the code execution was successful.

SCENARIO 2



The screenshot shows a Jupyter Notebook with a file named 'main.py'. The code in the notebook performs the following steps:

- Creates a DataFrame from a dictionary 'data'.
- Cleans the data by filling 'Name' with 'Unknown'.
- Converts 'Age' to numeric, coercing errors.
- Replaces 'Salary' values, specifically 'Sixty-Five Thousand' with 65000.
- Converts 'Salary' to integer type.
- Prints the missing values count.
- Prints the data types.

The output of the notebook shows a FutureWarning about the 'replace' method and the results of the missing values count and data types:

```
Missing Values Count:
```

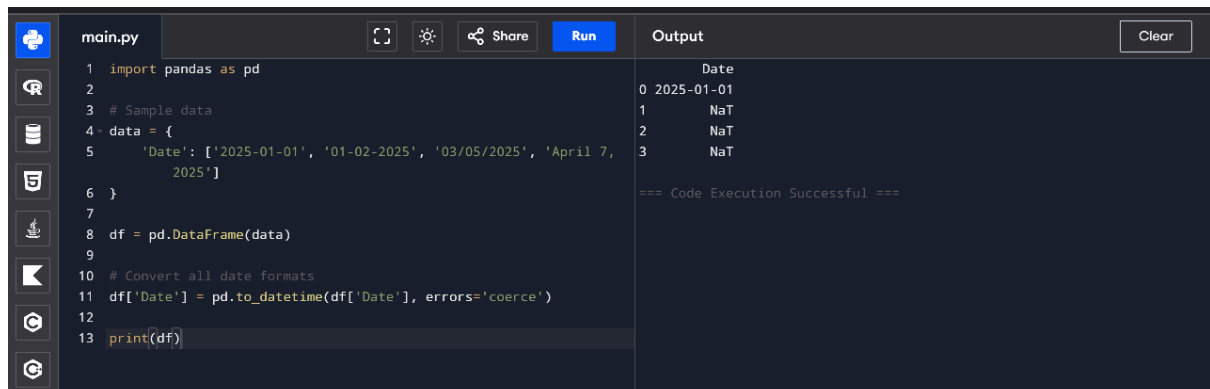
	Name	Age	Salary
0	0	2	0

```
Data Types:
```

	Name	Age	Salary
0	object	float64	int64

The output also indicates that the code execution was successful.

SCENARIO 3



```
main.py
1 import pandas as pd
2
3 # Sample data
4 data = {
5     'Date': ['2025-01-01', '01-02-2025', '03/05/2025', 'April 7, 2025']
6 }
7
8 df = pd.DataFrame(data)
9
10 # Convert all date formats
11 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
12
13 print(df)
```

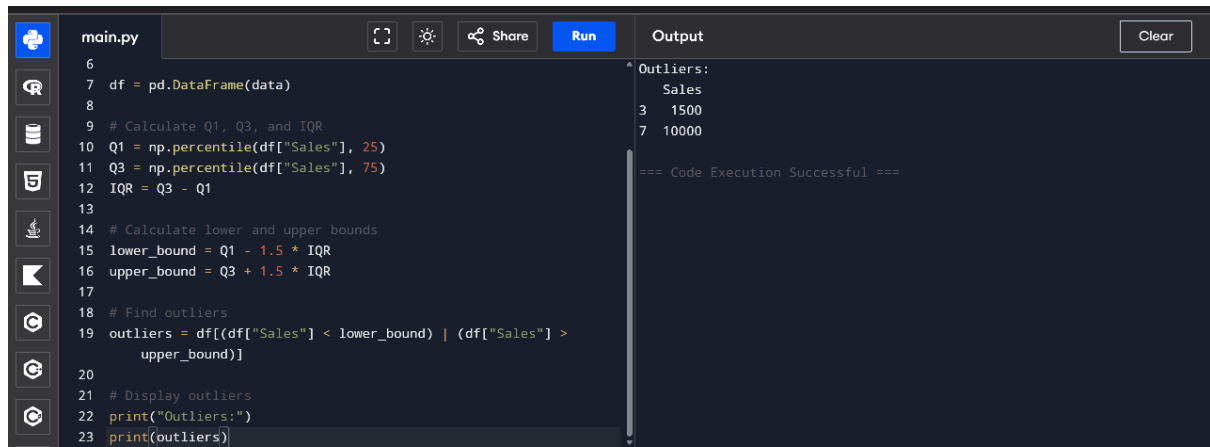
Output

```

      Date
0 2025-01-01
1      NaT
2      NaT
3      NaT

=== Code Execution Successful ===
```

SCENARIO 4



```
main.py
6
7 df = pd.DataFrame(data)
8
9 # Calculate Q1, Q3, and IQR
10 Q1 = np.percentile(df["Sales"], 25)
11 Q3 = np.percentile(df["Sales"], 75)
12 IQR = Q3 - Q1
13
14 # Calculate lower and upper bounds
15 lower_bound = Q1 - 1.5 * IQR
16 upper_bound = Q3 + 1.5 * IQR
17
18 # Find outliers
19 outliers = df[(df["Sales"] < lower_bound) | (df["Sales"] > upper_bound)]
20
21 # Display outliers
22 print("Outliers:")
23 print(outliers)
```

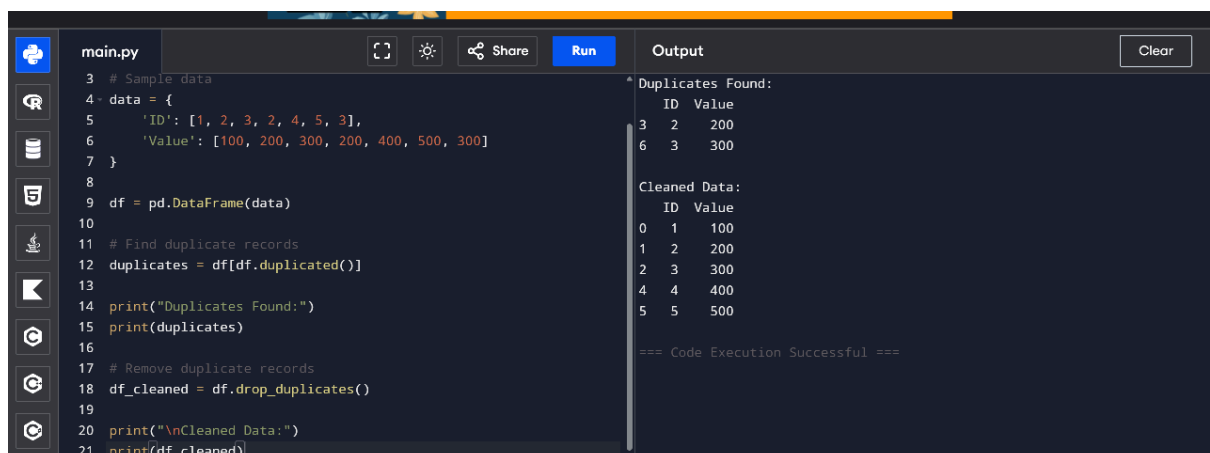
Output

```

* Outliers:
  Sales
3   1500
7  10000

=== Code Execution Successful ===
```

SCENARIO 5



```
main.py
3 # Sample data
4 data = {
5     'ID': [1, 2, 3, 2, 4, 5, 3],
6     'Value': [100, 200, 300, 200, 400, 500, 300]
7 }
8
9 df = pd.DataFrame(data)
10
11 # Find duplicate records
12 duplicates = df[df.duplicated()]
13
14 print("Duplicates Found:")
15 print(duplicates)
16
17 # Remove duplicate records
18 df_cleaned = df.drop_duplicates()
19
20 print("\nCleaned Data:")
21 print(df_cleaned)
```

Output

```

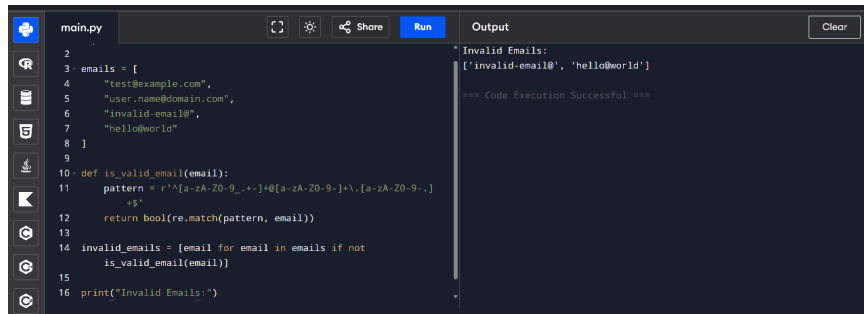
* Duplicates Found:
   ID  Value
3   2   200
6   3   300

Cleaned Data:
   ID  Value
0   1   100
1   2   200
2   3   300
4   4   400
5   5   500

=== Code Execution Successful ===
```

SCENARIO 6

SCENARIO 7

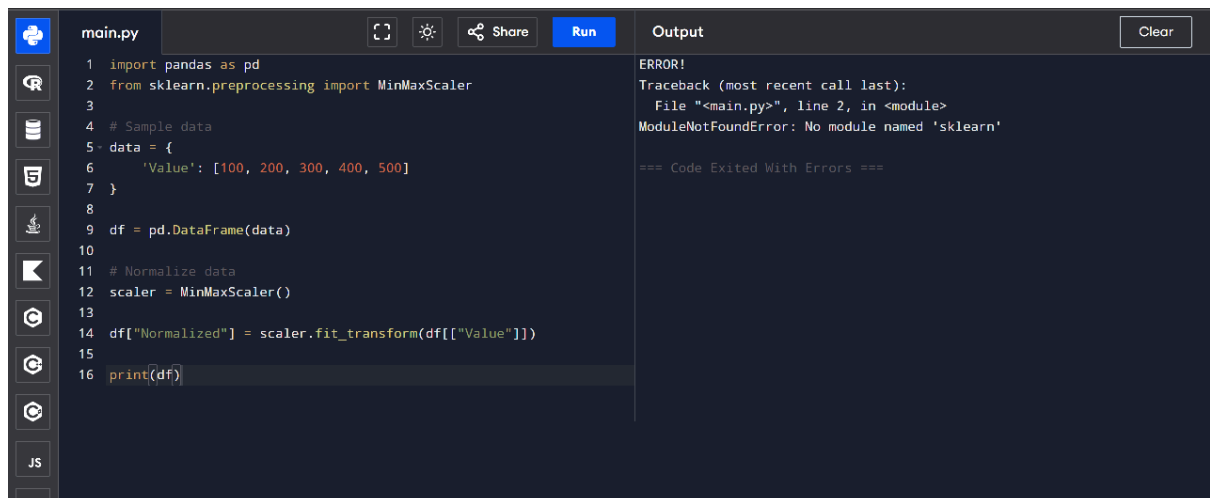


```
main.py
2
3 emails = [
4     "test@example.com",
5     "user.name@domain.com",
6     "invalid-email@",
7     "hello@world"
8 ]
9
10 def is_valid_email(email):
11     pattern = r'^[a-zA-Z0-9_+]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+'
12     return bool(re.match(pattern, email))
13
14 invalid_emails = [email for email in emails if not
15                   is_valid_email(email)]
16
17 print("Invalid Emails:")
```

Output

```
Invalid Emails:
['invalid-email@', 'hello@world']
=== Code Execution Successful ===
```

SCENARIO 8

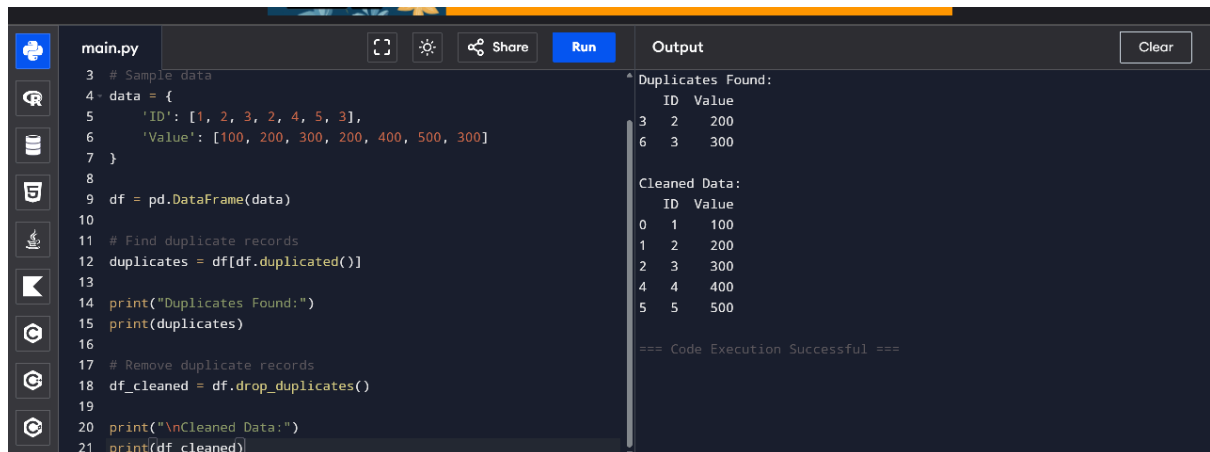


```
main.py
1 import pandas as pd
2 from sklearn.preprocessing import MinMaxScaler
3
4 # Sample data
5 data = {
6     'Value': [100, 200, 300, 400, 500]
7 }
8
9 df = pd.DataFrame(data)
10
11 # Normalize data
12 scaler = MinMaxScaler()
13
14 df["Normalized"] = scaler.fit_transform(df[["Value"]])
15
16 print(df)
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 2, in <module>
ModuleNotFoundError: No module named 'sklearn'
=== Code Exited With Errors ===
```

SCENARIO 9



The image shows a Jupyter Notebook interface with a file named 'main.py'. The code defines a dictionary 'data' with 'ID' and 'Value' lists. It creates a DataFrame, finds duplicates, prints them, and then removes them. The output shows the duplicates found and the cleaned data.

```
3 # Sample data
4 data = {
5     'ID': [1, 2, 3, 2, 4, 5, 3],
6     'Value': [100, 200, 300, 200, 400, 500, 300]
7 }
8
9 df = pd.DataFrame(data)
10
11 # Find duplicate records
12 duplicates = df[df.duplicated()]
13
14 print("Duplicates Found:")
15 print(duplicates)
16
17 # Remove duplicate records
18 df_cleaned = df.drop_duplicates()
19
20 print("\nCleaned Data:")
21 print(df_cleaned)
```

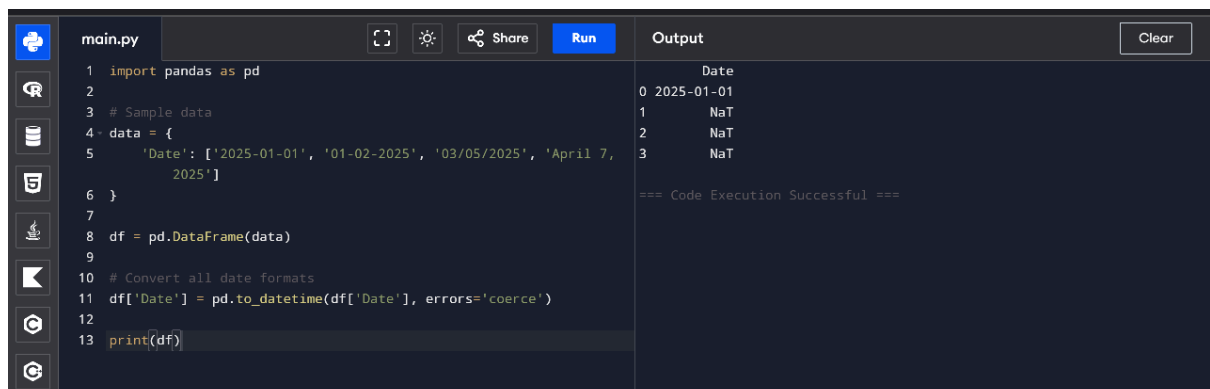
Output:

```
Duplicates Found:
  ID  Value
3  2    200
6  3    300

Cleaned Data:
  ID  Value
0  1    100
1  2    200
2  3    300
4  4    400
5  5    500

=== Code Execution Successful ===
```

SCENARIO 10



The image shows a Jupyter Notebook interface with a file named 'main.py'. The code imports pandas, defines a dictionary 'data' with a 'Date' list containing various date formats, creates a DataFrame, and converts all dates to a standard format. The output shows the resulting DataFrame.

```
1 import pandas as pd
2
3 # Sample data
4 data = {
5     'Date': ['2025-01-01', '01-02-2025', '03/05/2025', 'April 7, 2025']
6 }
7
8 df = pd.DataFrame(data)
9
10 # Convert all date formats
11 df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
12
13 print(df)
```

Output:

```
      Date
0 2025-01-01
1         NaT
2         NaT
3         NaT

=== Code Execution Successful ===
```